

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: ITA 0606

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO5: Introduction – NumPy Arrays – NumPy Basics- Math – Random – Indexing – Filtering – Statistics – Aggregation – saving data – Data analysis with Pandas – DataFrame – Combining – Indexing – File I/O – Grouping – Filtering – Sorting – Plotting
(BL 3)

Assessment Tool Weightage: 20%

QUESTIONS:15

Total Marks:25

Annexure A

Pseudocode Writing Task (Algorithm Design)

Code of Conduct:

*I Chalamala Sai Harshitha (192424346) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

C. Sai Harshitha

Annexure A

1. **Design a pseudocode algorithm to read a dataset from a file and preprocess it using array and DataFrame operations. The algorithm should handle missing values, remove duplicate records, normalize numerical columns, and standardize categorical values. Include steps for indexing, filtering invalid entries, and saving the cleaned dataset. Ensure the algorithm clearly defines input, processing steps, and output while maintaining computational efficiency.**

Aim

To design a pseudocode algorithm to read a dataset from a file and preprocess it using array and DataFrame operations by handling missing values, removing duplicate records, normalizing numerical columns, standardizing categorical values, indexing the data, filtering invalid entries.

Input

- Input dataset file.
- Numerical and categorical columns in the dataset.
- Output file path for saving the cleaned dataset.

Pseudocode

1. Start
2. Import NumPy and Pandas.
3. Read the dataset from the input file
DATA ← Read the input file into a DataFrame
4. Check whether the dataset is empty
IF DATA is empty THEN
Display "Dataset is empty"
STOP
END IF
5. Handle missing values
FOR each column in DATA
IF column is numerical THEN
Replace missing values with the median
ELSE IF column is categorical THEN
Replace missing values with the mode
END IF
END FOR
6. Remove duplicate records
DATA ← Remove duplicate rows from DATA

7. Standardize categorical values

FOR each categorical column

Remove leading and trailing spaces

Convert categorical values into a common format

END FOR

8. Identify numerical columns

FOR each numerical column

Convert the column into a NumPy array

END FOR

9. Normalize numerical columns

FOR each numerical column

MIN $\leftarrow$ Minimum value of the column

MAX $\leftarrow$ Maximum value of the column

IF MAX $\neq$ MIN THEN

NORMALIZED_VALUE $\leftarrow$

$(\text{VALUE} - \text{MIN}) / (\text{MAX} - \text{MIN})$

Replace the column with normalized values

ELSE

Set all values in the column to 0

END IF

END FOR

10. Apply indexing

Reset the DataFrame index after preprocessing

11. Filter invalid entries

Identify invalid or unwanted records

Remove rows containing invalid values

12. Validate the cleaned dataset

Check for remaining missing values and duplicate records

Check whether numerical values are properly normalized

13. Save the cleaned dataset

Write DATA to the specified output file

14. Display the number of records in the cleaned dataset

Output

A cleaned and preprocessed dataset saved to the specified output file, containing:

- No unwanted missing values. No duplicate records.
- Normalized numerical columns.

- Standardized categorical values.
- Properly indexed records. Invalid entries filtered out.
- A validated dataset ready for further data analysis.

Result

The dataset is successfully read, preprocessed, validated, and saved using array and DataFrame operations.

2. **Write a detailed pseudocode algorithm to compute statistical measures such as mean, median, standard deviation, and variance from a large dataset. The algorithm should demonstrate how to efficiently use array-based operations, apply filtering to exclude unwanted values, and perform aggregation. Include steps for validating the dataset and handling edge cases such as empty arrays or extreme outliers.**

Aim

To design a pseudocode algorithm to compute statistical measures such as mean, median, standard deviation, and variance from a large dataset using efficient array-based operations, filtering unwanted values, aggregation, dataset validation, and handling edge cases.

Input

- A dataset containing numerical values.
- Filtering conditions for excluding unwanted or invalid values.
- A large numerical array obtained from the dataset.

Pseudocode

1. Start
2. Read the dataset from the input file.
3. Extract the required numerical column
DATA ← Convert numerical values into an array
4. Validate the dataset
IF DATA does not exist THEN
Display "Invalid dataset"
STOP
END IF
5. Check for an empty array
IF length(DATA) = 0 THEN
Display "Array is empty"
STOP

END IF

6. Remove unwanted values

FILTERED_DATA ← Select only valid numerical values from DATA

Remove missing, infinite, and invalid values

7. Check the filtered array

IF length(FILTERED_DATA) = 0 THEN

Display "No valid values available"

STOP

END IF

8. Identify extreme outliers

Calculate the required statistical limits

Identify values that are extremely different from the normal range

9. Handle extreme outliers

Exclude unwanted extreme outliers from the calculation

Store the remaining values in CLEAN_DATA

10. Calculate the mean

SUM ← Aggregate all values in CLEAN_DATA

N ← Number of values in CLEAN_DATA & MEAN ← SUM / N

11. Calculate the median

Sort CLEAN_DATA

IF N is odd THEN

MEDIAN ← Middle value

ELSE

MEDIAN ← Average of the two middle values

END IF

12. Calculate the variance

Find the squared difference between each value and the MEAN

VARIANCE ← Average of the squared differences

13. Calculate the standard deviation

STANDARD_DEVIATION ← Square root of VARIANCE

14. Display the statistical measures:

Mean, Median, Variance, Standard Deviation

15. End

Output

The algorithm produces the following statistical measures from the validated dataset:

- Mean, Median, Variance, Standard Deviation

The calculations are performed after filtering unwanted values and handling edge cases such as empty arrays and extreme outliers.

Result

The statistical measures are successfully calculated from the valid dataset using efficient array-based operations, filtering, and aggregation.

3. Develop a pseudocode algorithm to combine two or more DataFrames containing related data. The algorithm must include steps for aligning indices, merging datasets based on a common key, removing duplicates, and validating the integrity of the merged data. Incorporate grouping and sorting operations, and ensure that constraints such as consistency and completeness of records are maintained.

Aim

To design a pseudocode algorithm to combine two or more related DataFrames by aligning indices, merging datasets using a common key, removing duplicate records, validating the integrity of the merged data, and performing grouping and sorting operations.

Input

- Two or more related DataFrames.
- A common key available in the DataFrames for merging.
- Relevant index values for aligning the DataFrames.

Pseudocode

1. Start
2. Import Pandas.
3. Read or create the required DataFrames:
 DATAFRAME1
 DATAFRAME2
 DATAFRAME3, if required
4. Validate the DataFrames:
 Check whether each DataFrame exists
 Check whether required columns are present
5. Align the indices:
 Ensure that the DataFrames have appropriate indices
 Reset or align indices wherever necessary
6. Identify the common key:
 COMMON_KEY ← Common column available in the DataFrames

7. Merge the DataFrames:

MERGED_DATA ← Merge DATAFRAME1 and DATAFRAME2 using
COMMON_KEY

IF additional DataFrames exist THEN

Merge each additional DataFrame using the appropriate common key

END IF

8. Remove duplicate records:

MERGED_DATA ← Remove duplicate rows

9. Validate merged data:

Check for missing values in important columns

Check whether the common key is consistent

Check whether records from the required DataFrames
are correctly included

10. Check completeness:

Compare the expected records with MERGED_DATA

Identify missing or unmatched records

11. Group the merged data:

GROUPED_DATA ← Group MERGED_DATA using the required category or key

12. Perform aggregation:

Calculate the required count, sum, average, or other aggregation for each group

13. Sort the results:

SORTED_DATA ← Sort GROUPED_DATA to the required column

14. Display the final merged, grouped, and sorted data.

15. End

Output

A merged, validated, grouped, and sorted DataFrame containing:

- Properly aligned indices.
- Records merged using the common key.
- Duplicate records removed.
- Consistent and complete records.
- Grouped and aggregated information.
- Sorted final results.

Result

The related DataFrames are successfully combined using the common key, validated for consistency and completeness, and organized through grouping and sorting operations.

4. Create a pseudocode algorithm that filters a dataset based on multiple conditions (e.g., value ranges, categories) and then sorts the filtered results. The algorithm should utilize indexing techniques, logical conditions, and efficient sorting methods. Include steps for handling large datasets and optimizing performance while ensuring that the output meets all filtering constraints.

Aim

To design a pseudocode algorithm to filter a dataset using multiple conditions such as value ranges and categories, apply indexing and logical conditions, and sort the filtered results efficiently while maintaining the required constraints.

Input

- Dataset containing numerical and categorical values.
- Numerical range conditions.
- Category-based filtering conditions.
- Column used for sorting.

Pseudocode

1. Start

2. Import NumPy and Pandas.

3. Read the dataset into a DataFrame:

DATA ← Read input dataset

4. Validate the dataset:

IF DATA is empty THEN

Display "Dataset is empty"

STOP

END IF

5. Identify the required columns:

NUMERICAL_COLUMN

CATEGORY_COLUMN

SORT_COLUMN

6. Apply indexing:

Use DataFrame indexing to access the required rows and columns

7. Define filtering conditions:

CONDITION1 ← NUMERICAL_COLUMN >= LOWER_LIMIT

CONDITION2 ← NUMERICAL_COLUMN <= UPPER_LIMIT

CONDITION3 ← CATEGORY_COLUMN belongs to the required categories

8. Combine the conditions:

```
FINAL_CONDITION ← CONDITION1 AND  
                  CONDITION2 AND  
                  CONDITION3
```

9. Filter the dataset:

```
FILTERED_DATA ← Select rows satisfying FINAL_CONDITION
```

10. Check the filtered result:

```
IF FILTERED_DATA is empty THEN  
    Display "No records satisfy the conditions"  
    STOP  
END IF
```

11. Sort the filtered results:

```
SORTED_DATA ← Sort FILTERED_DATA using SORT_COLUMN
```

12. For a large dataset:

Apply filtering before sorting to reduce the number of records that need to be sorted

13. Validate the final results:

Check that every record satisfies all filtering conditions

14. Display the filtered and sorted dataset.

15. End

Output

A filtered and sorted dataset containing only the records that satisfy all specified conditions, including:

- Required numerical value range and category or categories.
- Correct indexing. Proper sorting order.
- No records violating the filtering constraints.

Result

The dataset is successfully filtered using multiple logical conditions and indexing techniques, and the valid records are efficiently sorted to produce the required final output.

5. Write a pseudocode algorithm for a complete data analysis workflow that includes reading data from files, converting it into DataFrames, performing grouping and aggregation, and generating plots. The algorithm should specify steps for handling file input/output errors, applying transformations, and producing summarized results suitable for visualization. Ensure clarity in each stage from data loading to final output generation.

Aim

To design a pseudocode algorithm for a complete data analysis workflow that reads data from files, converts it into DataFrames, performs grouping and aggregation, applies transformations, handles file input/output errors, and generates plots for visualization.

Input

- Dataset stored in an input file.
- Required columns for analysis.
- Grouping column.
- Numerical column(s) for aggregation.
- Output file path for summarized results.
- Plotting requirements.

Pseudocode

1. Start
2. Import NumPy, Pandas, and plotting library.
3. Read the dataset:
 TRY
 DATA ← Read the input file
 CATCH file input error
 Display "Error: Unable to read the input file"
 STOP
END TRY
4. Check the dataset:
 IF DATA is empty THEN
 Display "Dataset is empty"
 STOP
END IF
5. Convert the data into a DataFrame:
 DATAFRAME ← Convert DATA into a DataFrame
6. Validate the DataFrame:
 Check column names
 Check data types
 Check for missing or invalid values
7. Handle missing or invalid values:
 Remove or replace missing values as required
 Remove invalid records
8. Apply data transformations:

Convert columns to appropriate data types

Create or modify required columns

Perform necessary numerical or categorical transformations

9. Group the data:

GROUPED_DATA ← Group DATAFRAME using the required grouping column

10. Perform aggregation:

SUMMARY ← Calculate required aggregate measures such as count, sum, mean

11. Sort the summarized results:

SUMMARY ← Sort SUMMARY according to the required column

12. Save the summarized results:

TRY

Write SUMMARY to the specified output file

CATCH file output error

Display "Error: Unable to save the output file"

STOP

END TRY

13. Generate plots:

Select appropriate columns from SUMMARY

Create a suitable plot

Add title, axis labels, and legend where required

14. Display the summarized results and plot.

15. Verify the final output:

Check whether the summary is complete

Check whether the plot represents the summarized data

16. End

Output

The workflow produces:

- A validated and transformed DataFrame.
- Grouped and aggregated summary results.
- A sorted summary dataset saved to an output file.
- A plot representing the summarized data for visualization.

Result

The complete data analysis workflow successfully reads the dataset, converts it into a DataFrame, performs required transformations, grouping, and aggregation, saves the summarized results, and generates a suitable visualization.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	25	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	15	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand